

Università
della
Svizzera
italiana

Faculty
of
Informatics

Bachelor Thesis

June 19, 2026

Bridging the Gap: Designing Decision-Support Tools for Small Hotel Management

Requirements Engineering, Design, and Evaluation of a Constraint-Solver Based Prototype

Theodor Vavassori

Abstract

Staff scheduling in small and medium-sized hotels looks simple from the outside, yet managers handle it almost entirely with spreadsheets, with no dedicated software support to speak of. Existing tools are either too costly, too complex, or built for enterprise contexts that bear little resemblance to the flexible, resource-constrained reality of an independent property. This thesis presents the requirements engineering, design, and evaluation of a prototype decision- support system built to close that gap. Following a requirements-first methodology, seven semi-structured interviews were conducted with hotel managers in northern Italy, six of which were retained and tagged into around forty candidate features, clustered into three design principles: standalone operation, eighty per cent automation with manager approval, and native support for hospitality shift patterns. These principles shaped a three-tier prototype: a React frontend, a Spring Boot backend running the Timefold constraint solver, MongoDB Atlas for persistence, and an AI-powered occupancy import module built with Spring AI and the Anthropic Claude API. The prototype generates constraint- based schedules, allows transparent manual override, and automates monthly payroll reporting. A System Usability Scale evaluation with five participants from the interview cohort yielded a mean score of seventy-eight point five, in the Good range, with four of the five participants scoring Good to Excellent.

Advisor
Dr. Andrea Mocci

Advisor's approval (Dr. Andrea Mocci): Date:

Contents

1 Introduction **3**
- 1.1 Motivation . . . 3
- 1.2 Approach . . . 3
- 1.3 Contributions . . . 4
- 1.4 Document Structure . . . 4

2 State of the Art **4**
- 2.1 Staff Scheduling as a Constraint Problem . . . 4
- 2.2 Decision Support and Mixed-Initiative Interaction . . . 5
- 2.3 User-Centred Design . . . 5

3 Requirements Elicitation **5**
- 3.1 Why Interviews . . . 6
- 3.2 Interview Design and Conduct . . . 6
- 3.3 Key Findings from the Interviews . . . 7
- 3.4 The Tagging Methodology . . . 8
- 3.5 Feature Analysis and Clustering . . . 9
- 3.6 Design Principles Derived from the Analysis . . . 9
- 3.7 Survey of Existing Tools . . . 10

4 Architecture, Design, and Development **10**
- 4.1 Architectural Overview . . . 11
- 4.2 Technology Choices and Their Rationale . . . 11
- 4.3 Domain Model . . . 12
- 4.4 Schedule Lifecycle . . . 14
- 4.5 Constraint Model . . . 14
- 4.6 REST API Design . . . 16
- 4.7 Schedule Generation and the Occupancy Engine . . . 16
- 4.8 AI-Powered Occupancy Import . . . 17
- 4.9 Records and Payroll Layer . . . 18
- 4.10 Frontend Design . . . 18
- 4.11 Development Process . . . 21

5 Evaluation **21**
- 5.1 Evaluation Goals . . . 21
- 5.2 The System Usability Scale . . . 21
- 5.3 Feedback Instrument Design . . . 21
- 5.4 Evaluation Design: Self-Administered Sessions . . . 22
- 5.5 Participants . . . 22
- 5.6 Results . . . 23
  - 5.6.1 SUS Scores . . . 23
  - 5.6.2 Feature Relevance . . . 23
  - 5.6.3 Schedule Quality Comparison . . . 24
  - 5.6.4 Adoption Intention . . . 24
  - 5.6.5 Qualitative Feedback . . . 24
- 5.7 Discussion . . . 24
  - 5.7.1 The Main Cohort: Four Participants in the Good to Excellent Range . . . 24
  - 5.7.2 P4: A Useful Outlier . . . 24
  - 5.7.3 Feature Relevance Interpretation . . . 25
  - 5.7.4 Adoption Intention and Schedule Quality . . . 25
  - 5.7.5 Limitations . . . 25

6 Conclusion **25**
6.1 Summary of Contributions 25
6.2 Reflections on the Approach 26
6.3 Limitations 26
6.4 Future Work 27
6.5 Final Remarks 27

Project Artifacts **27**

1 Introduction

1.1 Motivation

Small and medium-sized hotels rarely use dedicated software to build their staff schedules. Most rely on a spreadsheet, rebuild or copied over from one week to the next, and adjusted by hand to account for availability, requested days off, and expected occupancy. This is not because no scheduling software exists. It is because the tools on the market do not offer a good balance between the features a small hotel actually needs and the complexity and cost of adopting them. A system is suitable, in this context, if it provides real scheduling value without requiring an investment in price, integration or training that an independent property cannot justify.

Staff scheduling in small and medium-sized hotels is one of those operational problems that sounds simple from the outside but turns out to be a tangle of constraints, informal rules, and domain knowledge that is very difficult to fully formalise. Hotels run continuously, demand fluctuates sharply with the season, staff contracts are heterogeneous, and the workforce spans departments such as reception, restaurant, housekeeping, and kitchen, each with its own shift patterns and coverage requirements [18, 9]. The hospitality industry is also characterised by high turnover and a high proportion of seasonal, part-time, and fixed-term workers [8], which means the person building the schedule rarely has the luxury of a stable, well-known team.

Large hotel chains handle this complexity with enterprise resource planning platforms that cost thousands of euros per month and require dedicated HR staff to configure and maintain. Independent hotels, which make up the majority of accommodation properties in many regions of Europe, including the Ligurian Riviera where this research was conducted, have no equivalent. The tools available to them either do too much, forcing adoption of payroll, HR, and POS modules they do not need, or too little, offering nothing more than a digital version of the same blank calendar they already have in Excel [20, 17].

The academic scheduling literature offers no shortage of frameworks supporting optimisation algorithms for staff scheduling [25, 4, 14]. But optimisation algorithms are not end-user products. A hotel manager cannot configure a library implementing a constraint solver. What the literature has not produced, at least not for this segment of the market, is a tool that is actually ready to use, that speaks the language of hospitality operations, and that respects the manager's need to remain in control of the final schedule.

This project is an attempt to build that tool, starting from the requirements.

1.2 Approach

The project followed a requirements-first methodology. Nothing about the system was designed until the people who would use it had been asked, at length, what they actually needed.

Seven semi-structured interviews were conducted with hotel directors and general managers operating independent properties in northern Italy. Six were retained for analysis after one was excluded due to incomplete data. The interviewees managed properties ranging from small family-run hotels to multi-structure complexes with over a hundred seasonal staff. All of them scheduled their workforce manually, mostly with Excel, sometimes with paper. None used any dedicated scheduling software, for reasons they explained in considerable detail.

The interviews were recorded, transcribed, and systematically tagged using a coding scheme developed specifically for this project and grounded in requirements engineering methodology [26, 10, 34]. Every line of every transcript was read and annotated. The result was a set of tagged documents that captured hard constraints, soft preferences, pain points, decision rules, workflow descriptions, and verbatim statements, all linked back to the specific interview that produced them.

From this evidence base, approximately forty candidate features were identified. These were then clustered and prioritised by hand into a final set of around twenty requirements that defined the scope of the prototype. Three design principles emerged from this process and shaped every subsequent decision.

Standalone first. The system must be useful on its own, without requiring integration with any other software. The most commonly cited reason interviewees gave for not adopting existing tools was the setup cost and forced bundling.

80/20 automation. The system generates approximately 80% of the schedule automatically. The manager reviews, adjusts, and confirms the rest. No schedule is ever published without explicit approval. This was the consensus model across all six interviews: managers did not want a system that removed their agency, they wanted one that removed the mechanical work.

Hospitality-native shift model. The tool must natively support split shifts, non-standard weekly structures, and seasonal staffing patterns. The most commonly cited structural failure of existing scheduling software was that it assumed standard eight-hour, Monday-to-Friday shifts, a model that describes almost no hotel in existence.

These principles shaped the design of a web-based decision-support prototype [2, 30]: a system that generates constraint-satisfying schedules automatically, surfaces violations transparently through warnings, and allows the manager to override any assignment without friction.

1.3 Contributions

The work produced four contributions.

- A structured requirements analysis of SME (Small and Medium Enterprises) hotel scheduling needs, grounded in six interviews with expert practitioners, documented using a replicable tagging methodology, and synthesised into a prioritised feature set with explicit traceability to the interview evidence.
- A survey of existing scheduling tools and a clear account of why they are not adopted in the SME hotel context, going beyond cost to identify the structural mismatches in automation model, flexibility, and hospitality fit.
- A working prototype implementing the core requirement set: automatic constraint-based schedule generation, occupancy-driven staffing, manual override with full transparency, and a monthly reporting layer for payroll reconciliation. The system is publicly deployed and was used in the evaluation.
- A usability evaluation using the System Usability Scale [12], conducted with participants also drawn from the original interview cohort, assessing whether the prototype meets the usability bar that hotel managers would need to actually adopt it.

1.4 Document Structure

The remainder of this report is organised as follows. Section 2 covers the background literature relevant to the scheduling problem in the hospitality domain and to decision support in general. Section 3 describes the requirements elicitation process in full: the interview methodology, the tagging scheme, the feature clustering, and the survey of existing tools. Section 4 describes how the prototype was built: the architecture, technology choices, data model, constraint implementation, API, frontend design, and deployment. Section 5 presents the evaluation: the SUS instrument, the evaluation protocol, the participants, and the results. Section 6 reflects on what was learned, what the prototype does not yet do, and what would come next.

2 State of the Art

This chapter reviews the literature relevant to this project across three areas: the staff scheduling problem and how it has been studied (Section 2.1), the concept of mixed-initiative decision support and why it is the right framing for this domain (Section 2.2), and the user-centred design methodology that grounds the requirements elicitation work described in the following chapter (Section 2.3).

2.1 Staff Scheduling as a Constraint Problem

The general scheduling problem asks: given a set of workers, a set of shifts, a planning horizon, and a collection of rules about who can work when, produce an assignment that satisfies all mandatory rules and does a reasonable job of satisfying the preferences. In formal terms it is a constraint satisfaction and optimisation problem belonging to the class of NP-hard problems, meaning there is no algorithm that solves it efficiently in the general case [22, 28].

A large body of research has addressed this problem, producing solution approaches based on constraint programming [14], evolutionary and metaheuristic algorithms [4, 32], mathematical programming [19], and neural network based methods [29]. The vast majority of this work has focused on nurse rostering in healthcare settings, which has become the de facto benchmark for workforce scheduling research due to the availability of standardised problem instances and the institutional interest in improving hospital staffing [25, 11].

Hotel and hospitality scheduling has received comparatively little attention, despite being arguably harder in several respects. Hospitals have fixed departments and relatively stable staffing pools. Hotels have departments with radically different shift patterns: a restaurant runs on morning, afternoon, and evening splits, reception runs continuous coverage, and housekeeping peaks on checkout days. They also experience high seasonal variation that can triple or halve the workforce in a matter of weeks, and a workforce characterised by high turnover, short-term contracts, and part-time arrangements [18, 8].

The work that does exist in the hotel scheduling space points consistently to the same findings. Murakami et al. [24] show that demand-responsive staffing, calculating required headcount from expected guest volume rather than using

fixed numbers, can significantly reduce labour costs while maintaining service quality. Malony [20] demonstrates using discrete event simulation that staffing decisions directly affect service efficiency during high-occupancy periods. Hur et al. [17] note that real-time schedule adjustment is frequent and consequential in hospitality operations: managers deviate from generated schedules regularly, and a good system needs to support that rather than resist it. Mohan [23] finds that maximising employee satisfaction through preference accommodation has measurable effects on retention and quality of work, which matters particularly in an industry with structurally high turnover.

The scheduling problem in a small hotel is not primarily a hard algorithmic challenge. The number of workers is modest, typically between ten and a hundred, the planning horizon is short (one week at a time usually), and the constraint set, while complex, is finite and describable. What makes it difficult for a hotel manager doing it manually is the volume of interdependencies that have to be held in mind simultaneously: who has already worked five days this week, who asked for Saturday off, whether the breakfast team has enough seniority on Sunday, and the absence of any system that checks these things automatically.

2.2 Decision Support and Mixed-Initiative Interaction

Fully automated scheduling systems exist and can produce mathematically optimal solutions. They are also frequently rejected by the managers who are supposed to use them.

The reasons for this are well documented. Managers have contextual knowledge that no model captures: they know that one particular worker should not be scheduled alongside another, that a specific employee is going through a difficult personal situation, that the booking system underestimates arrivals on long weekends. They also have accountability: the schedule is their responsibility, and a system that generates it opaquely and does not let them override it easily is not a tool, it is a liability [3].

The concept of mixed-initiative interaction addresses this directly. Coined in the AI planning literature [2] and applied to scheduling systems by Smith et al. [30] and others [33, 21], it describes a model in which both the human and the system can take initiative, with the balance of control shifting based on the situation. In practice, for a scheduling tool, this means that the system generates a candidate schedule, the manager reviews it, overrides anything that does not work, and the system respects those corrections absolutely. Constraint violations are shown as warnings, not blocking errors. The human is always in the loop.

Butler et al. [13] provide a detailed case study of this philosophy applied to a real-world scheduling context and find that success depends on aligning the system closely with users' existing mental models and workflows. People who already have a way of doing something will reject a tool that forces them to do it differently, even if the new way is technically superior. This finding anticipates precisely what the interview phase of this project confirmed: managers did not want their scheduling heuristics replaced, they wanted the mechanical work removed instead.

2.3 User-Centred Design

User-centred design [1, 15] provides the broader methodological framework within which this project operated. Rather than deriving system requirements from technical capabilities or prior literature alone, user-centred design grounds specifications in empirical study of the intended users: their tasks, their environments, their goals, and their constraints. It is particularly appropriate when designing for a user group that has no prior experience with dedicated software in the problem domain, and whose requirements are therefore not well described in existing literature.

Beyer and Holtzblatt's contextual design methodology [10] operationalises this approach through a sequence of steps: contextual inquiry (interviewing and observing users at work), work modelling (abstracting the data into structured representations), consolidation (identifying patterns across multiple users), and design (deriving system features from the consolidated models). The requirements elicitation process used in this project follows this structure, adapted to the constraints of a thesis-scale project.

The distinction between hard and soft constraints, which forms the backbone of the tagging scheme described in the following chapter, is grounded in the requirements engineering literature [34, 31]. Hard constraints are non-negotiable, in that violating them renders a solution infeasible, while soft constraints express preferences that the system should satisfy where possible but may trade off against coverage requirements.

3 Requirements Elicitation

This chapter describes the requirements engineering process that grounded the design and development of the prototype. It covers the interview methodology (Section 3.1, Section 3.2), the key findings that emerged from the interviews (Section 3.3), the tagging scheme used to analyse the transcripts (Section 3.4), the feature extraction and clustering process (Section 3.5), the design principles derived from the analysis (Section 3.6), and a survey of existing tools (Section 3.7). Together, these activities produced the specification that drove every subsequent design decision.

3.1 Why Interviews

Requirements engineering is the process of finding out what a software system needs to do and documenting it in a form that can drive design and development [26, 31]. Among the available elicitation techniques, semi-structured interviews are one of the most commonly used for capturing domain-specific and contextual knowledge from experts in complex, tacit-knowledge-heavy domains [16, 27].

Hotel scheduling is exactly that kind of domain. The rules that govern a schedule, such as who can work when, how many people are needed on a given shift, and which workers need certain pairings with colleagues, are rarely written down. They live in the head of the manager who builds the schedule every week. A questionnaire or a requirements template would not surface all of this knowledge. An interview, conducted by someone who has read enough background literature and has experience in the field to ask the right questions, has a much better chance.

A semi-structured interview uses a prepared guide, a set of questions and topics, but doesn't follow it rigidly. The interviewer can follow unexpected threads, probe ambiguous answers and adapt the conversation to what the specific participant knows and cares about. This flexibility is essential when the interviewee pool is as diverse as hotel managers. a receptionist-turned-general-manager of a 25-room family hotel has a very different perspective on scheduling than a manager overseeing three four-star properties with 130 seasonal staff, and the interview guide needs to leave room for both [7].

3.2 Interview Design and Conduct

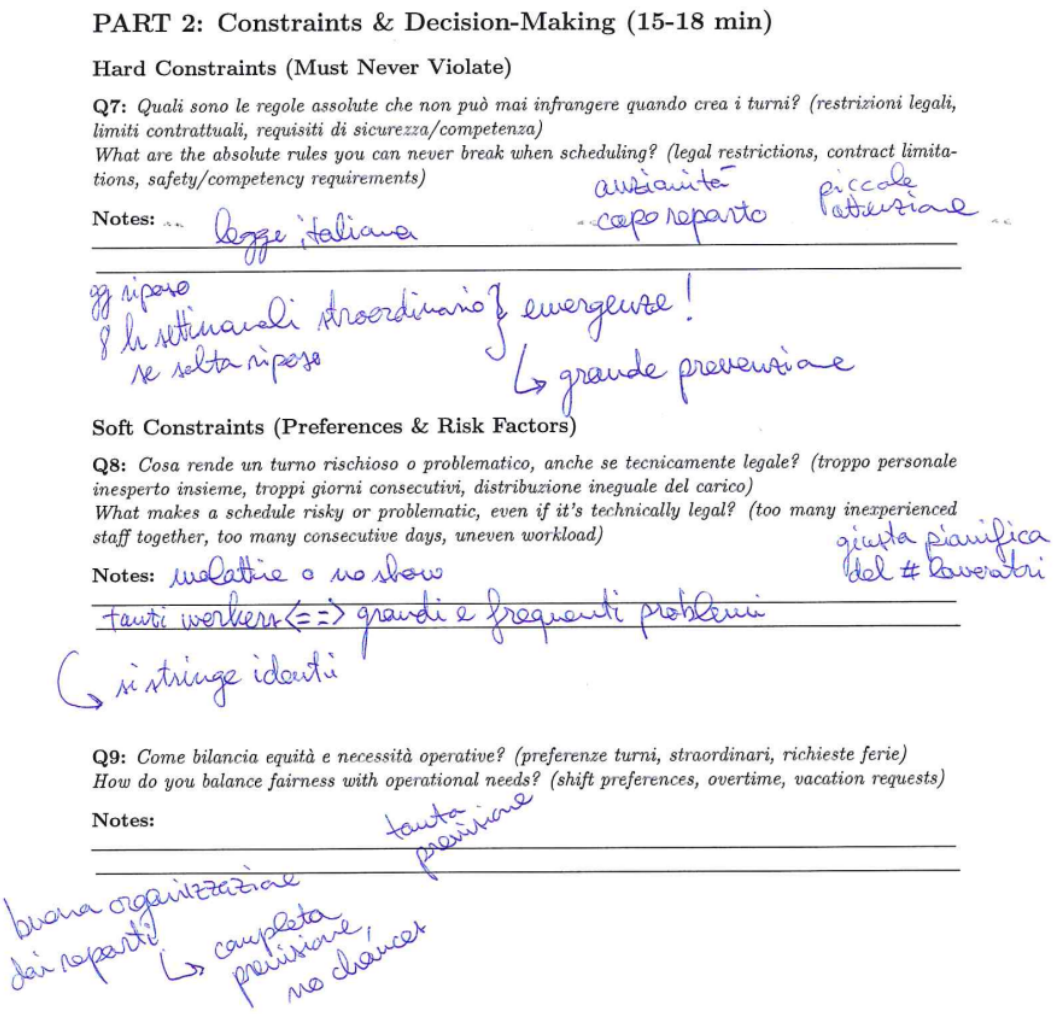
Seven interviews were conducted for this project. Six were retained for analysis; one was excluded after producing insufficient usable data. The interviewees were hotel directors and general managers at independent properties in the Ligurian Riviera in Italy. They managed properties ranging from small boutique hotels to multi-structure resort complexes. All of them currently schedule their staff manually, and none used dedicated scheduling software.

The interview script was prepared in advance, in Italian, and covered the following areas in order:

- The interviewee's role and the size and type of their property.
- How the current scheduling process works end-to-end: when it happens, who does it, how long it takes, what inputs are required.
- Which tools are currently used and why (or why not dedicated software).
- What rules and constraints govern scheduling, both legal and informal.
- What causes problems, errors, or extra work in the current process.
- What data the manager has access to (PMS exports, occupancy forecasts, time-clock records) and how they currently use it in scheduling.
- What they would want from a scheduling tool if one existed, and what they would refuse to accept.

The final question, on what they would refuse to accept, proved to be among the most informative in the entire script. It surfaced the three non-negotiable design constraints that shaped the entire prototype: the system must work standalone without requiring integration with other software; it must allow the manager to override any assignment; and it must support hospitality-native shift patterns rather than assuming a standard Monday-to-Friday, eight-hour model.

All interviews were conducted in Italian, audio-recorded with the participant's consent, and transcribed in full. Identifying information about the properties was anonymised before analysis. Interviews lasted between 25 and 45 minutes. Figure 1 shows an excerpt from one of the anonymised transcripts.


PART 2: Constraints & Decision-Making (15-18 min)

Hard Constraints (Must Never Violate)

Q7: *Quali sono le regole assolute che non può mai infrangere quando crea i turni? (restrizioni legali, limiti contrattuali, requisiti di sicurezza/competenza)*
What are the absolute rules you can never break when scheduling? (legal restrictions, contract limitations, safety/competency requirements)

Notes: legge italiana — assistente capo reparto — piccola attenzione

gg riposo
8 h settimanali straordinario → emergenze!
se salta riposo
↳ grande prevenzione

Soft Constraints (Preferences & Risk Factors)

Q8: *Cosa rende un turno rischioso o problematico, anche se tecnicamente legale? (troppo personale inesperto insieme, troppi giorni consecutivi, distribuzione ineguale del carico)*
What makes a schedule risky or problematic, even if it's technically legal? (too many inexperienced staff together, too many consecutive days, uneven workload)

Notes: malattie o no show — giusta pianifica del # lavoratori

tanti workers <=> grandi e frequenti problemi
↳ si stringe identi

Q9: *Come bilancia equità e necessità operative? (preferenze turni, straordinari, richieste ferie)*
How do you balance fairness with operational needs? (shift preferences, overtime, vacation requests)

Notes: tanta previsione

buona organizzazione dei reparti
↳ completa previsione, no chaos

Figure 1. Excerpt from one of the interviews.

3.3 Key Findings from the Interviews

While the full analysis is encoded in the tagged transcripts and the feature list, several findings recurred consistently enough across interviews to be worth stating directly.

Excel is universal. Every single interviewee builds their schedule in a spreadsheet. The spreadsheet is rebuilt from scratch each week, or copied from the previous week and modified. This is not a preference, it is a symptom of the absence of anything better.

The occupancy calculation is already being done manually. Multiple interviewees had developed their own formulas in Excel to translate expected guest numbers into required headcount for each shift. One interviewee described a detailed workload model: rooms multiplied by an estimated service time, divided by shift duration, giving a minimum headcount. The fact that this calculation was being performed manually, in a spreadsheet, by multiple independent managers, was a strong validation signal that the occupancy engine needed to be a first-class feature of the prototype.

Seniority and pairing rules are universal. Every interviewee had informal rules about which workers could be scheduled together. Most commonly: at least one senior worker on every shift, no all-junior evenings, and specific workers who should not be assigned to the same shift for personal or performance reasons. These rules are never written down and never enforced automatically by current tools.

Seasonal complexity is the norm, not the exception. Several interviewees managed properties with fewer than ten workers in winter and over eighty in peak summer. Their scheduling process changes completely between seasons, and any tool that does not support this range is useless to them.

End of month payroll reconciliation is a significant pain point. Every interviewee described some version of an end of month process: comparing planned hours to actual hours, calculating overtime, extracting data for payroll.

This was almost universally done by hand, often taking several hours, and was a source of errors. Several interviewees explicitly asked for a tool that could maintain these records automatically.

Trust and control are prerequisites, not preferences. No interviewee was willing to adopt a system that made scheduling decisions they could not reverse or inspect. The phrase "I need to be the one who decides" or its equivalent appeared in every single interview. This was the single most consistent finding and had the largest influence on the design.

3.4 The Tagging Methodology

Raw interview transcripts are not requirements. Turning them into requirements requires a systematic analytical step: reading the transcripts carefully, identifying the significant content, and classifying it in a way that makes patterns visible across multiple interviews and produces evidence that is traceable back to its source.

For this project, a coding scheme was developed before analysis began, drawing on established concepts from requirements engineering and contextual inquiry [26, 10, 34, 31]. The scheme defines ten tag types, each with a precise definition and domain-specific examples:

- **`[CONSTRAINT-HARD]`** a non-negotiable rule that any valid schedule must satisfy [34]. Typically legal or contractual: maximum weekly working hours, mandatory rest periods between shifts, minimum daily rest. Violating a hard constraint is not an acceptable trade-off.
- **`[CONSTRAINT-SOFT]`** — a preference to be respected where possible but overrideable when coverage requires it. Examples: worker shift type preferences, seniority balance across shifts, informal agreements about rest days.
- **`[FEATURE-NEEDED]`** — an explicit or implicit request for system functionality. Sometimes stated directly ("I would want it to show me the total hours per worker"); sometimes inferred from a pain point ("I always have to re-count the hours by hand" implies a need for automatic hour tracking).
- **`[PAIN-POINT]`** — a source of friction, error, or frustration in the current manual process [10]. Pain points are particularly valuable because they identify precisely where a system can create real, measurable value.
- **`[DECISION-RULE]`** — an informal heuristic the manager applies when building a schedule that would not appear in any written policy. Examples: always pair at least one senior worker with juniors on evening service; never assign two new hires to the same reception shift; give the best and/or most important shifts to the most reliable workers first.
- **`[CURRENT-TOOL]`** — references to tools currently in use: Excel, paper rosters, WhatsApp groups, PMS systems, badge-in time-tracking devices.
- **`[WORKFLOW]`** — descriptions of how the scheduling process currently unfolds: when it happens, who is involved, what physical or digital artifacts are produced, how the schedule is communicated to staff.
- **`[INTEGRATION]`** — references to external systems or data sources that the scheduling process depends on or interacts with: the PMS, the payroll system, time-clock data, the HR system.
- **`[RISK-FACTOR]`** — situations identified as operationally risky when they appear in a schedule: an all-junior night shift, a department understaffed during a fully-booked weekend, a worker scheduled for six consecutive days.
- **`[QUOTE]`** — a verbatim statement of particular evidential value, useful for grounding design decisions in direct evidence from the field.

Each transcript was tagged line by line. A single passage could carry multiple tags: a statement like "I always re-count the hours every Sunday evening because I've been burned by overtime before" would receive `[PAIN-POINT]`, `[WORKFLOW]`, and `[FEATURE-NEEDED]` simultaneously. Every tagged item was accompanied by a brief note in English explaining its significance.

AI assistance was then used at the feature extraction stage. Once all six transcripts were fully tagged, AI cross-referenced the tagged material to surface recurring patterns across interviews. This step is methodologically comparable to using qualitative analysis software such as ATLAS.ti or NVivo. The intellectual work, namely deciding what to tag, how to interpret a passage, and how to resolve apparent contradictions between interviews, remained entirely human. The AI accelerated the cross-referencing step across a volume of tagged text that would have been impractical to scan manually for patterns. The output was approximately forty candidate features, each with explicit links to the tagged passages that motivated it.

3.5 Feature Analysis and Clustering

Forty candidate features is too many for a single-developer bachelor project prototype. Not all of them are equally important and not all of them are equally feasible to implement in a few months. The clustering step was done to address both dimensions simultaneously.

Each candidate feature was evaluated on two axes: importance, how frequently and urgently the underlying need was expressed across the interviews, and implementation difficulty, estimated as development effort withing the available timeline. Features were then grouped by functional area into six clusters and ordered by priority. This process refined the scope to approximately twenty features covering the first five clusters below.

Cluster A, Employee Foundation. Staff profiles with contract type, contracted weekly hours, maximum daily hours, seniority level, shift type preferences, and a designated weekly rest day. This is the foundational data layer: every scheduling decision the system makes depends on knowing who the workers are and what their constraints are. It also includes the active/inactive toggle for soft delete, which allows seasonal workers and workers on leave to be excluded from scheduling without losing their historical records.

Cluster B, Scheduling Core. Automatic generation of a constraint-satisfying weekly schedule given the employee data and shift definitions, manual override of any individual assignment and a schedule life cycle with distinct Draft, Generated, Published and Finalized states. This is the central value proposition of the system. The 80/20 principle lives here: the system fills in the schedule automatically based on the defined constraints and the manager reviews, corrects and approves the result.

Cluster C, Occupancy Engine. Demand-responsive headcount calculation. Rather than hard-coding a fixed number of workers per shift, the manager defines how many guests one worker can serve, and the system computes the required staffing level from the day's expected occupancy data. Occupancy data can be entered manually per day or imported automatically from a PMS export via the AI extraction feature. This cluster was among the most strongly validated in the interviews. Multiple managers had already implemented this logic manually in Excel, confirming that the need was real and widespread.

Cluster D, Records and Payroll. Automatic maintenance of worker hour records when a schedule is finalised. For each worker, the system records the exact shifts worked, total minutes, rest days and an estimated cost for the week. These records are aggregated monthly into a reporting view that shows per-worker and per-department totals, and exported as a PDF for handover to payroll or accounting. This cluster addresses one of the most consistently mentioned pain points across all interviews: the manual end of month reconciliation process that every manager described performing from scratch, typically taking several hours and prone to errors.

Cluster E, Warnings and Visibility. Real-time constraint violation detection surfaced inline on the schedule grid. Unassigned shifts slots shown in a dedicated row, highlighted in red, so the manager cannot miss a coverage gap. Overtime warnings for workers whose assigned hours approach or exceed their contracted cap. This cluster implements the transparency that the mixed-initiative model requires: the system silently accepts a problematic schedule while always telling the manager what is wrong and why.

Cluster F, Deferred Features. Fairness-based rotation, ensuring undesirable shifts are distributed equitably over time, skill and competency tagging, direct payroll software export, and PMS API integration. These features were identified as requirements from the interviews but slightly adjusted due to certain constraints. They represent in part the natural next development phase.

3.6 Design Principles Derived from the Analysis

Three non-negotiable design principles emerged from the interview analysis and governed every subsequent decision in the project.

Standalone first. The system must provide immediate value without requiring any integration with other software. Integrations with PMS platforms, payroll systems or HR tools are additive features, never prerequisites. The most commonly cited reason interviewees gave for not adopting existing scheduling tools was the setup cost and forced bundling with modules they did not want. A tool that cannot be used in isolation will not be evaluated at all.

80/20 automation. The system generates approximately 80% of the schedule automatically, while the manager reviews, adjusts and confirms is the remaining 20%. No schedule is published or finalised without explicit manager approval at every step. This was the consensus model across all six interviews. What managers rejected was automation that removed their ability to correct and approve what is generated.

Hospitality-native shift model. The tool must natively support split shifts, non-standard weekly structures, such as a six-day weeks and 1.5-day rest patterns, and seasonal staffing swings. The most commonly cited structural failure of existing scheduling software was that it assumed standard eight-hour Monday-to-Friday shifts, a model that describes almost no hotel in existence.

3.7 Survey of Existing Tools

A survey of existing scheduling software was conducted in parallel with the interview analysis, serving two purposes: to establish what the target users have already tried and rejected, and to provide an evidence base for the design decisions that differentiate this prototype from what already exists. Three categories of tools were examined.

Integrated workforce management platforms. The dominant products in this category are Fourth[1] (which acquired HotSchedules), Humanity [2] (now part of TCP Software), and Deputy[3]. They are designed for medium-to-large hospitality groups and are genuinely capable systems, handling scheduling, payroll, time and attendance, POS integration, and PMS connectivity in a unified platform.

The problem is the adoption model. Pricing is per employee per month, and full scheduling functionality requires higher-tier subscriptions. For a hotel with twenty staff, the monthly cost is difficult to justify when the current process takes two hours with Excel. More critically, these platforms require all-or-nothing adoption: accessing the scheduling module means adopting the payroll, HR, and POS modules as well. For an independent hotel that already has a payroll provider and a PMS it is happy with, this is simply not an option. The integration complexity, including migrating data from existing systems, retraining staff, and managing API connections, far exceeds the operational capacity of most SME operators.

The automation model is also the wrong one for the SME context. These platforms either generate schedules fully automatically, which interviewees explicitly said they would not trust, or provide blank digital calendars with no intelligence at all, which offer nothing over a spreadsheet.

Standalone SaaS scheduling tools. Products like When I Work[4], Planday[5], and Staffomatic[6] occupy a lower price point and focus specifically on scheduling rather than full HR management. They are simpler to set up and more affordable for small teams.

But they are generic. Their constraint models assume standard shift patterns: fixed start and end times, simple weekly cycles, no occupancy-driven staffing logic. Hospitality-specific features like split shifts, seasonal headcount variation and demand-responsive staffing are absent or rudimentary. The configuration overhead is still significant: a manager must spend hours mapping their actual workflow into the tool's data model before anything useful comes out. Several interviewees had tried one of these tools at some point and abandoned it. The reasons were consistent: it did not understand the shift structure of their hotel, it could not handle their seasonal patterns and it took longer to configure than it saved in scheduling time.

Open-source constraint solvers. Frameworks like Timefold Solver[7] represent the state of the art in constraint-based scheduling. They can express complex combinations of hard and soft constraints, handle large planning horizons, and optimise across multiple objectives simultaneously.

They are engineering frameworks, not end-user products. Using Timefold to build a hotel scheduling system requires a developer to model the domain, implement constraint rules, build a persistence layer and design a user interface from scratch. There is no hospitality-specific configuration, no guided setup and no out-of-the-box interface. A hotel manager cannot use it, and in the current market there is no product that wraps it into something they could.

The barriers identified across all three categories form a consistent picture: cost and pricing model, forced integration with unwanted modules, automation that removes manager agency, a shift model that does not match how hotels actually operate, technical complexity that requires developer involvement and setup overhead that exceeds the time the tool saves. The prototype developed in this project was designed from the ground up to avoid some of these barriers.

4 Architecture, Design, and Development

This chapter describes how the prototype was built. It covers the architecture and technology choices (Section 4.1, Section 4.2), the data model (Section 4.3), the schedule life cycle (Section 4.4), the constraint model (Section 4.5), the REST API (Section 4.6), the solver and occupancy engine (Section 4.7), the AI powered occupancy import (Section 4.8), the records and payroll layer (Section 4.9), the frontend design (Section 4.10), and the development process (Section 4.11). The goal throughout is not just to describe what was built, but to explain why, connecting every significant decision back to the requirements that motivated it.

[1] See `https://www.fourth.com`.
[2] See `https://tcpsoftware.com/products/humanity/`.
[3] See `https://www.deputy.com`.
[4] See `https://wheniwork.com`.
[5] See `https://www.planday.com`.
[6] See `https://staffomatic.com`.
[7] See `https://timefold.ai`.

4.1 Architectural Overview

The prototype is a standard three tier web application, shown in Figure 2. A React single-page application runs in the user's browser. It talks to a SpringBoot backend via a REST API. The backend stores everything in a Mongo DB Atlas cloud database and calls the Anthropic API through Spring AI for occupancy data extraction. Nothing about this architecture is surprising and that is deliberate: the goal was to spend development time solving the scheduling problem, not the architecture to be implemented.

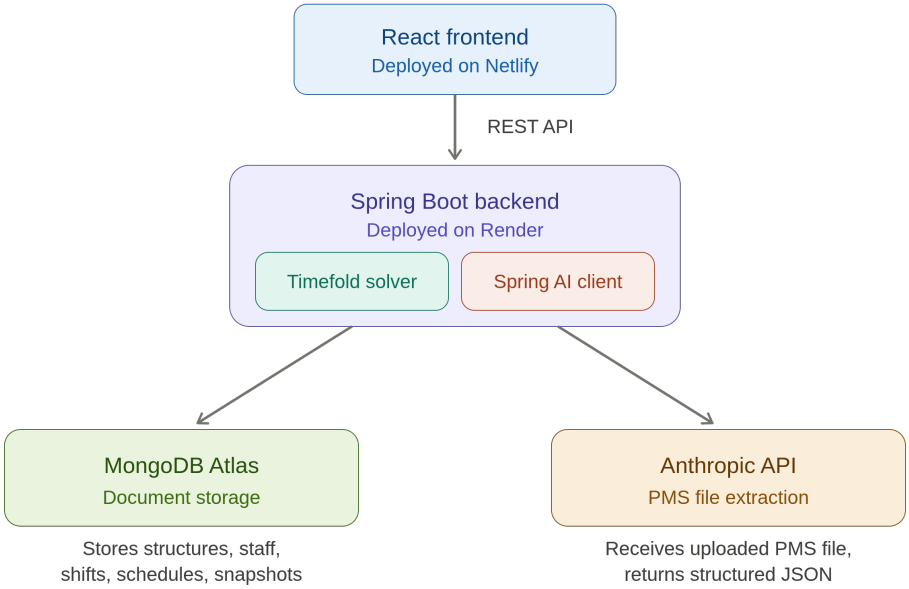


Figure 2. Overview of the three tier architecture: the React frontend, the Spring Boot backend hosting the Timefold solver and the Spring AI client, MongoDB Atlas for persistence, and the Anthropic API for PMS file extraction.

The separation between frontend and backend pays off in a specific way. The REST API is the boundary between the UI and the constraint logic. If the frontend needs further development or changes, the API stays the same. The solver and the constraint model are entirely server-side, so the browser never touches them directly.

The system is deployed publicly. The backend runs on Render's[8] free tier. The frontend is hosted on Netlify's[9] free tier. Both are accessible from any browser, which was a practical requirement for the remote evaluation sessions. Access is protected by authentication and the login credentials were issued to evaluation participants directly.

4.2 Technology Choices and Their Rationale

Every technology choice in the stack can be traced to a specific reason.

Java 21 and Spring Boot 3. Java 21 is the current long-term support release of the language. Spring Boot 3[10] was chosen because it makes the routine parts of building a REST API fast and predictable: auto-configuration handles the boilerplate, Spring Data MongoDB handles persistence, and Spring Web MVC provides the routing layer. Hot-reload via Spring DevTools kept the development loop short. Spring Boot is also widely used in enterprise Java development, making the choice academically defensible with excellent documentation.

MongoDB Atlas. The domain model involves entities with nested structures. A `Staff` record contains a list of shift type preferences and a list of unavailable dates. A relational database would handle these with join tables; MongoDB Atlas[11] stores them as embedded documents, making queries simpler and the schema easier to evolve. During development, requirements changed: new fields were needed, old ones renamed, and MongoDB's schema flexibility meant no migration scripts were required. Atlas's free tier also meant the same cloud instance served development, testing, and the live deployment.

[8] See `https://render.com`.
[9] See `https://www.netlify.com`.
[10] See `https://spring.io/projects/spring-boot`.
[11] See `https://www.mongodb.com/atlas`.

One trade-off worth noting: MongoDB in this project's scope does not offer multi-document ACID transactions as it is in its default configuration. For a production payroll system this would require careful design. For a prototype whose purpose is to validate requirements, it is an acceptable limitation for the time being.

Timefold Solver. The decision to use a dedicated constraint solver was one of the most important in the project. The alternative, writing a custom scheduling algorithm, would have consumed the majority of development time on a problem entirely orthogonal to the thesis's contribution. The contribution of this project is in the requirements engineering, the interface design, and the occupancy model. The solver is an engine, and using an existing, well-tested one is the right call.

Timefold Solver[12] version 1.17.0 is the successor to OptaPlanner, following the original core team's spin-out from Red Hat in 2023 [25]. It provides full Spring Boot 3.x support via a dedicated Maven starter dependency. Its `ConstraintStreams` API expresses scheduling rules as composable stream operations over the planning entities, which maps well to the way the requirements were documented: each tagged constraint from the interview analysis becomes one or two lines of solver constraint code, as shown in 1 in Section 4.5.

Spring AI and the Anthropic API. One feature that emerged consistently from the interviews was the need to import occupancy data from a Property Management System. Every hotel that uses a PMS can export that data, but the export format varies by vendor: some produce PDFs, some CSVs, some Excel files. Rather than building a separate parser for each possible format, the prototype uses Spring AI[13] to call the Anthropic Claude API[14] and extract the relevant numbers from whatever file the manager uploads. The implementation and its design rationale are described in detail in Section 4.8.

React and Vite. React[15] was chosen for the frontend because its functional component model maps naturally to the tabbed departmental view that emerged from the UI design process. Each tab is an independent component managing its own state via React hooks: `useState` for data and UI state, `useEffect` for data loading on mount, and `useRef` for DOM access in the print functionality. Vite[16] was preferred over Create React App for its significantly faster development server and hot-reload speed.

4.3 Domain Model

The domain model was derived directly from the feature clustering described in Section 3.5. Six primary entities were identified as necessary and sufficient for the prototype scope, plus one record-keeping entity for the payroll layer. Figure 3 shows a simplified class diagram of the model.

[12] See `https://timefold.ai`.
[13] See `https://spring.io/projects/spring-ai`.
[14] See `https://www.anthropic.com`.
[15] See `https://react.dev`.
[16] See `https://vite.dev`.

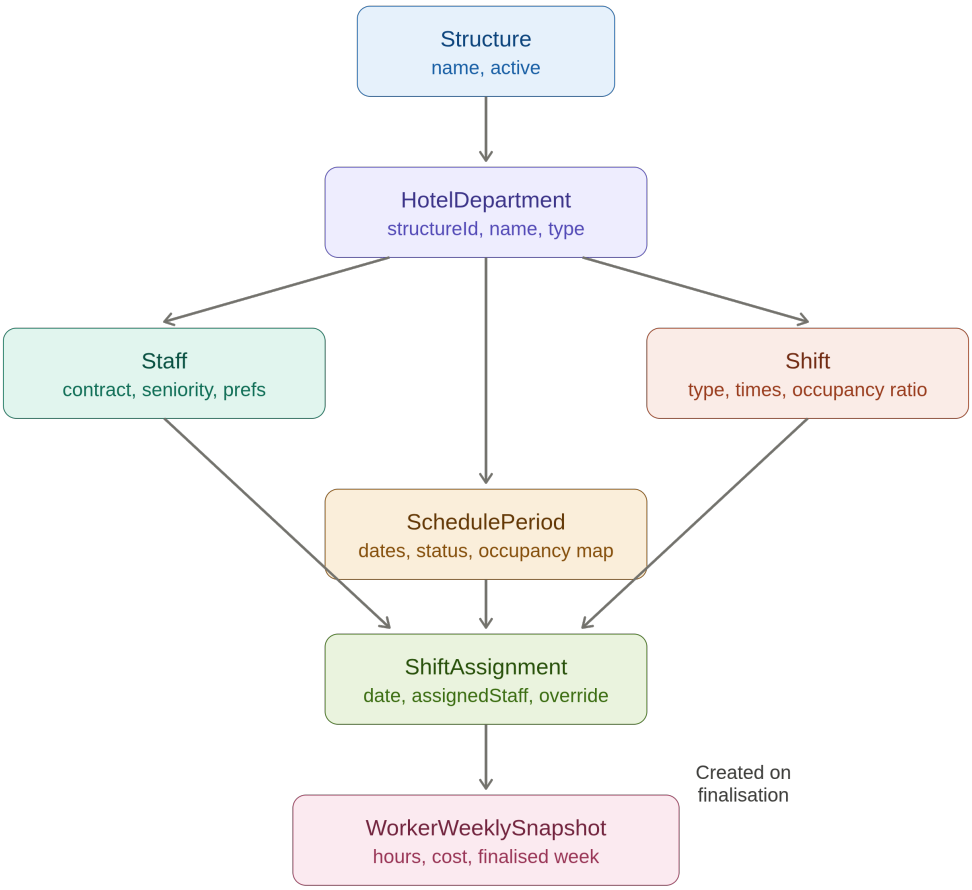


Figure 3. Simplified class diagram of the domain model, showing the seven primary entities and their relationships. `ShiftAssignment` is the Timefold planning entity; `WorkerWeeklySnapshot` is created automatically on finalisation.

Structure. A hotel property. The entry point for everything else. A structure has a name and an `active` flag implementing soft delete. All other entities carry the parent `structureId`, meaning a single deployment can support multiple independent properties.

HotelDepartment. A department within a structure: Restaurant, Reception, Housekeeping, Kitchen, and so on. Departments are typed with a `departmentType` enum but carry a freeform `name` for display. Departments are independently scheduled: each has its own staff pool, shift templates, and schedule periods. This reflects the reality confirmed by every interviewee: the restaurant manager builds the restaurant schedule, the reception supervisor builds the reception schedule, and the two processes do not interact.

Staff. A hotel employee. Each record carries: first name, last name, contract type (`FULL_TIME`, `PART_TIME`, `ON_CALL`), contracted weekly hours, a maximum daily duration (configurable in hours or minutes, defaulting to eight hours if unset), seniority level (`JUNIOR`, `STANDARD`, `SENIOR`), a designated weekly rest day, monthly salary, a list of preferred shift types, and the `structureId`/`departmentId` scoping. Staff are soft-deleted: toggling inactive removes them from the scheduling pool without removing historical records. This matters for payroll: a worker on extended leave is still present in the monthly report for the weeks they worked.

Shift. A shift template within a department. A shift has a name, a type (`MORNING`, `AFTERNOON`, `EVENING`, `SPLIT`), start and end times, and three occupancy-related fields: the metric it is associated with (e.g. total guests, rooms occupied, breakfast/lunch/dinner covers, or combined restaurant covers), the guests per worker ratio, and a default minimum headcount. Rather than hard-coding a fixed staffing number, the system derives required headcount dynamically from actual expected guest volume. Several interviewees had built this exact calculation manually in Excel.

SchedulePeriod. A planning horizon, typically one week. It has a start date, an end date, and a life cycle status. Occupancy data is stored separately as one `DailyOccupancyRecord` document per structure per day and is queried by date range when the solver needs it (**??**).

ShiftAssignment. The unit of scheduling and the Timefold planning entity. Each `ShiftAssignment` links a shift, a date, and an assigned staff member (the planning variable). A `manualOverride` boolean marks assignments changed by the manager after generation, distinguishing them visually in the UI from solver-generated assignments. In the

current implementation, overrides are not yet preserved across a subsequent re-solve of the same period; this is noted as a limitation in Section 6.3.

WorkerWeeklySnapshot. The permanent record-keeping entity, described fully in Section 4.9.

4.4 Schedule Lifecycle

A schedule period moves through five distinct states, each representing a meaningful milestone in the scheduling workflow, as shown in Table 1.

Status	What it means	What the manager can do
`DRAFT`	Period created, no schedule yet	Edit period, trigger generate
`SOLVING`	Solver is running	Wait, or cancel
`GENERATED`	Solver finished, schedule ready	Override assignments, publish
`PUBLISHED`	Schedule confirmed and visible	View, finalise
`FINALIZED`	Schedule locked permanently	View only, export reports

Table 1. The five states of the schedule life cycle and the manager actions available at each stage.

The transitions are explicit actions: the manager triggers generation, reviews the result, publishes when satisfied, and finalises when the week is complete. No automatic transition occurs without manager approval. The `FINALIZED` status triggers the snapshot mechanism (Section 4.9) and is irreversible; a confirmation modal prevents accidental finalisation.

Because the solver can take several seconds to run, the frontend does not block waiting for the result. Instead, after sending the solve request, it polls the backend every three seconds, checking whether the period's status has changed from `SOLVING` to `GENERATED`. A cancel button is shown during polling; pressing it sends a cancellation request to the backend and returns the period to `DRAFT`. This polling approach keeps the interface responsive and gives the manager an escape hatch if the solver takes unexpectedly long.

4.5 Constraint Model

The constraints implemented in the prototype were derived directly from the tagged transcripts. Every constraint corresponds to one or more `[CONSTRAINT-HARD]` or `[CONSTRAINT-SOFT]` tags in the interview data.

Hard constraints (a solution violating any of these is infeasible):

- *No double assignment.* A worker may not be assigned to two different shifts on the same day. Tagged as a hard constraint in every interview; a legal requirement under Italian labor law.
- *Weekly hours cap.* Total assigned minutes per week must not exceed the worker's contracted weekly hours. Exceeding this without overtime authorization is a labor law violation.
- *Daily hours cap.* Total assigned minutes in a single day must not exceed the worker's `maxDailyMinutes`. Configurable per worker to accommodate different contract types.
- *Department eligibility* (`departmentEligibility`). A worker may only be assigned to shifts in departments listed in their eligibility list. New staff are pre-populated with their primary department as their sole eligible department at creation time; if the eligibility list is left empty, the constraint does not restrict the worker to any particular department.
- *Weekly rest day* (`respectDayOff`). A worker may not be assigned a shift on their designated weekly day off.

Soft constraints (violations are penalized but the solution remains feasible):

- *Senior presence per shift* (`seniorPresencePerShift`). A reward is given when at least one senior-level worker is present on a given shift and date.
- *Junior never alone* (`juniorNeverAlone`). A junior-level worker assigned to a shift with no senior-level worker present on the same shift and date is penalised, encoding the decision rule that came up in every interview: a junior should not be left without senior supervision.
- *Minimum shift coverage* (`minimumCoveragePerShift`). Unassigned slots for a given shift and date are penalised in proportion to how many remain unfilled.

- *Unassigned slot penalty* (`penaliseUnassignedSlots`). Every individual unassigned slot carries an additional fixed penalty, making the solver strongly prefer any valid assignment over leaving a slot empty.

- *Preferred shift type* (`preferredShiftType`). A reward is given when a worker is assigned to a shift type that appears in their list of preferred shift types.

The `manualOverride` flag is recorded on each assignment when the manager reassigns it, and is surfaced in the UI to distinguish manager-corrected shifts from solver-generated ones. The constraint provider does not currently special-case these assignments during re-solving; carrying overrides forward across a re-solve is left as future work (Section 6.3).

1 shows the real Timefold `ConstraintStreams` implementation of two representative constraints from the prototype's `ScheduleConstraintProvider`: the hard weekly hours cap and the soft rule penalising a junior worker scheduled without senior supervision on the same shift.

```
private Constraint weeklyHoursCap(ConstraintFactory factory) {
    return factory.forEach(ShiftAssignment.class)
        .filter(a -> a.getAssignedStaff() != null && a.getShift() != null)
        .groupBy(
            a -> a.getAssignedStaff(),
            ConstraintCollectors.sum(a -> (int) Duration.between(
                a.getShift().getStartTime(),
                a.getShift().getEndTime()).toMinutes()))
        .filter((staff, totalMinutes) ->
            totalMinutes > staff.getContractedHoursPerWeek() * 60)
        .penalize(HardSoftScore.ONE_HARD,
            (staff, totalMinutes) -> totalMinutes
                - staff.getContractedHoursPerWeek() * 60)
        .asConstraint("Weekly hours cap exceeded");
}

private Constraint juniorNeverAlone(ConstraintFactory factory) {
    return factory.forEach(ShiftAssignment.class)
        .filter(a -> a.getAssignedStaff() != null
            && a.getShift() != null
            && a.getAssignedStaff().getSeniorityLevel() == SeniorityLevel.JUNIOR)
        .ifNotExists(ShiftAssignment.class,
            Joiners.equal(ShiftAssignment::getShift),
            Joiners.equal(ShiftAssignment::getDate),
            Joiners.filtering((junior, other) -> other.getAssignedStaff() != null
                && other.getAssignedStaff()
                    .getSeniorityLevel() == SeniorityLevel.SENIOR))
        .penalize(HardSoftScore.ONE_SOFT, a -> 10)
        .asConstraint("Junior working without senior supervision");
}
```

Listing 1. Two constraints from the prototype's `ScheduleConstraintProvider`: the hard weekly hours cap, which penalises in proportion to the number of minutes a worker is over-scheduled, and the soft rule penalising a junior worker scheduled with no senior present on the same shift and date.

The full constraint provider defines ten constraints in total: five hard constraints covering double assignment, daily and weekly hours caps, department eligibility, and weekly rest day respect; and five soft constraints covering senior presence, junior supervision, minimum shift coverage, unassigned slot penalties, and preferred shift type rewards. `ShiftAssignment` is registered as the Timefold planning entity, and the overall planning problem is wrapped in a single `ScheduleSolution` class, shown in 2, annotated with `@PlanningSolution`. The `Staff` list is the problem fact collection and value range provider (the pool of workers the solver can assign), the `ShiftAssignment` list is the planning entity collection (the variables the solver fills in), and the `HardSoftScore` field holds the resulting score once a solution is found.

```java
@Data
@NoArgsConstructor
@PlanningSolution
public class ScheduleSolution {

    @ProblemFactCollectionProperty
    @ValueRangeProvider
    private List<Staff> staffList;

    @PlanningEntityCollectionProperty
    private List<ShiftAssignment> assignments;

    @PlanningScore
    private HardSoftScore score;

}
```

Listing 2. The `ScheduleSolution` class, annotated with Timefold's `@PlanningSolution`, wrapping the staff pool, the planning entity list, and the resulting hard/soft score.

4.6 REST API Design

The REST API follows a hierarchical resource structure mirroring the domain model's scoping relationships, as show in Table 2.

Resource	Base path
Structures	`/api/structures`
Departments	`/api/structures/{sId}/departments`
Staff	`/api/structures/{sId}/departments/{dId}/staff`
Shifts	`/api/structures/{sId}/departments/{dId}/shifts`
Schedule periods	`/api/structures/{sId}/departments/{dId}/schedule-periods`
Solve	`/api/schedule/{periodId}/solve`
Cancel solve	`DELETE /api/schedule/{periodId}/solve`
Assignments	`/api/schedule/{periodId}/assignments`
Override	`/api/schedule/assignments/{id}/override`
Publish	`/api/structures/{sId}/departments/{dId}/schedule-periods/{pId}/publish`
Finalise	`/api/structures/{sId}/departments/{dId}/schedule-periods/{pId}/finalize`
Occupancy extraction (AI preview)	`/api/structures/{sId}/occupancy/extract`
Occupancy confirmation (save)	`/api/structures/{sId}/occupancy/confirm`
Monthly report	`/api/structures/{sId}/reports/monthly?year=&month=`

Table 2. The REST API resource hierarchy, mirroring the structure and department scoping of the domain model.

Standard HTTP verbs throughout. All controllers return JSON. Deletes are soft: they toggle the `active` flag rather than removing records. CORS is configured globally to allow requests from the frontend domain.

4.7 Schedule Generation and the Occupancy Engine

When the manager triggers schedule generation, the following sequence runs on the backend.

The `ScheduleSolverService` loads all active staff and shift templates scoped to the target department. For each day in the period and each shift, it calculates the required headcount. If occupancy data is available for the shift's associated metric, the headcount is $\lceil \text{occupancyValue}/\text{guestsPerWorker} \rceil$, floored at the shift's default minimum. If no occupancy data is available for that day, the default minimum is used.

The required number of `ShiftAssignment` entities is created with null `assignedStaff` values. Each solve run rebuilds the full set of assignment slots from the current staff and shift data; a mechanism for carrying forward prior manual overrides was not implemented in this iteration. The assembled `SchedulePlan`, the Timefold `@PlanningSolution` object, is handed to the solver, which optimises for minimum hard constraint violations first, then soft constraint penalty. The solved assignments are persisted to MongoDB, the period status is updated to `GENERATED`, and the result is returned to the frontend.

4.8 AI-Powered Occupancy Import

Getting occupancy data into the system is a practical bottleneck. Hotel managers have this data, their PMS tracks guests, arrivals, departures, and covers daily, but export formats vary by vendor: some produce PDFs, some CSVs, some Excel files. Building a dedicated parser for each format would be brittle and expensive to maintain.

The prototype takes a different approach: it uses Spring AI to call the Anthropic Claude API and extract the occupancy numbers from whatever file the manager uploads. Spring AI's `ChatClient` is configured with an Anthropic API key and a structured extraction prompt, shown in 3.

The uploaded file is sent to the model as a document, and the model is asked to return the daily guest metrics, namely breakfast covers, lunch covers, dinner covers, arrivals, departures, and total guests, as a structured JSON object keyed by date.

```
private static final String SYSTEM_PROMPT = """
            You are a data extraction assistant for hotel management software.
            Your job is to extract daily occupancy and meal data from hotel PMS exports.

            The file may be from any PMS system (Scidoo, Opera, Mews, Protel, etc.)
            and may use any language (Italian, English, French, German, Spanish, etc.).

            [...]

            Return ONLY a valid JSON array. Each element must have exactly these fields:
            {
              "date": "YYYY-MM-DD",
              "totalGuests": <integer or null>,
              "arrivingGuests": <integer or null>,
              "departingGuests": <integer or null>,
              "stayingGuests": <integer or null>,
              "roomsOccupied": <integer or null>,
              "breakfastCovers": <integer or null>,
              "lunchCovers": <integer or null>,
              "dinnerCovers": <integer or null>,
              "occupancyRate": <decimal 0.0-1.0 or null>
            }

            Rules:
            - One element per calendar day
            - Use null for any field not present in the document
            - Do not include totals/summary rows -- daily rows only
            - Do not include any text before or after the JSON array
            - Do not wrap in markdown code blocks
            - Dates in DD/MM/YYYY format must be converted to YYYY-MM-DD
            """;
```

Listing 3. Structured extraction prompt sent to the Anthropic Claude API via Spring AI's `ChatClient`, alongside the uploaded PMS export file (some of it has been redacted for readability purposes in the report).

The extracted data is returned to the frontend as a preview before anything is saved. Fields where the extraction was uncertain and returned `NULL` are highlighted in amber. The manager reviews the values, corrects anything that looks wrong and saves explicitly. This human-in-the-loop confirmation step applies the same mixed-initiative principle to data import as to the schedule itself: the system does the extraction work, the manager validates the result.

Once confirmed, the occupancy data is stored as `DailyOccupancyRecord` documents scoped to the structure and date, and is immediately available to the solver for any schedule period covering that date. The uploaded file is not stored server-side after extraction; only the extracted values are persisted. A GDPR privacy notice is shown prominently in the UI explaining this.

This feature directly addresses one of the most tedious steps in the current weekly workflow: transcribing occupancy figures from a PMS into a scheduling spreadsheet. The AI import turns that transcription task into a review-and-confirm task, which is significantly faster and less error-prone.

4.9 Records and Payroll Layer

One of the clearest and most consistently expressed pain points in the interviews was the end of month payroll reconciliation. Every manager described the same process: at the end of the month, they sit down with the schedule history and manually reconstruct how many hours each worker worked, week by week, calculate overtime, and produce a summary for payroll or accounting. This takes several hours, is done entirely by hand, and is a frequent source of errors.

The prototype addresses this with a dedicated record-keeping layer built around the `WorkerWeeklySnapshot` entity. A snapshot is a permanent, immutable record of a single worker's working week: it captures every shift they were assigned, the exact duration of each, total minutes and shifts worked, rest days in the period, and an estimated weekly cost computed from the worker's contract type and an hourly rate snapshot at the moment of finalisation. Snapshotting the hourly rate at finalisation time is an important design decision: it means historical cost records remain correct even if a worker's salary is updated later.

Snapshots are created automatically when a schedule period is finalised. The `WorkerSnapshotService` loads all assignments for the period, groups them by staff member, computes the shift entries, off days, and cost, and saves one snapshot per worker. The process is idempotent: if a snapshot already exists for a given worker and period (identifiable by the `schedulePeriodId/staffId` pair), it is skipped. This means re-triggering finalisation is safe and produces no duplicate records.

One subtlety worth noting: workers who are inactive at the time of finalisation are still snapshotted if they have assignments in the period. A worker might be toggled inactive during a week, due to illness for example, but have worked the earlier days. Their record must still appear in the payroll report.

Monthly aggregation is computed on the fly by the reporting endpoint rather than stored as a separate document. The endpoint queries all snapshots for a given `structureId`, `year`, and `month`, groups them by department and by worker, and sums hours, shifts, rest days, and cost. This produces the data for the reports tab without requiring any additional write operations during finalisation.

The reports tab at the structure level shows: a per-worker horizontal bar chart of hours worked in the selected month; a structure summary section with combined totals across all departments; per-department breakdowns with individual worker rows, their weekly shift detail, and monthly totals; and a PDF export button that opens a print-optimised version in a new browser tab. This export is intended to be handed directly to the hotel's payroll provider or accounting team, replacing the manual summary that every interviewee was producing by hand.

4.10 Frontend Design

The frontend is built around three levels of navigation mirroring the domain model. The home screen shows hotel structures as cards. Clicking a structure opens a department list. Clicking a department opens the department page with three tabs: Schedule, Staff, and Shifts.

The **Schedule tab**, shown in Figure 4, is the primary interface. It shows a Staff × Day grid: one row per worker, one column per day in the planning period. Each cell shows the shift pills assigned to that worker on that day. Unassigned slots are collected into a dedicated row at the bottom, highlighted in red. Manually overridden shifts are shown in amber with a "Manual" badge. Clicking any shift pill opens an inline dropdown for reassignment; the override is applied immediately without a page reload. A cost and hours summary panel above the grid shows estimated weekly cost and per-worker hour totals with warnings for workers approaching or exceeding their contracted cap.

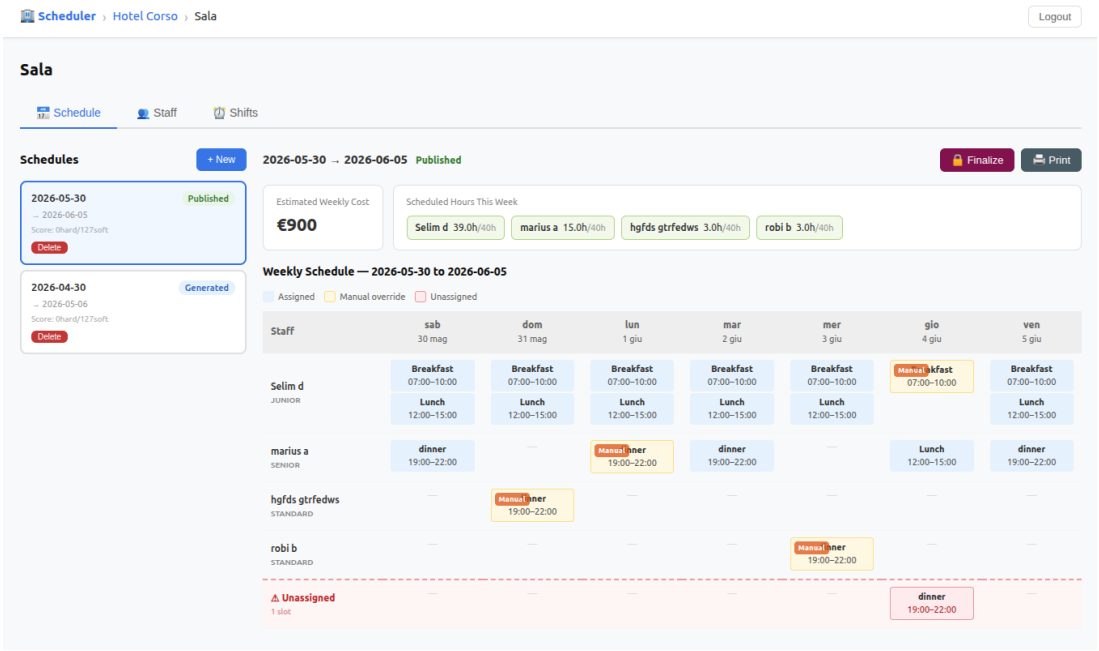


Figure 4. The Schedule tab, showing the Staff × Day grid with assigned, manually overridden, and unassigned shifts.

The **Staff tab**, shown in Figure 5, provides full CRUD management of the department's staff pool, including all contract fields, seniority, shift preferences, and the active/inactive toggle.

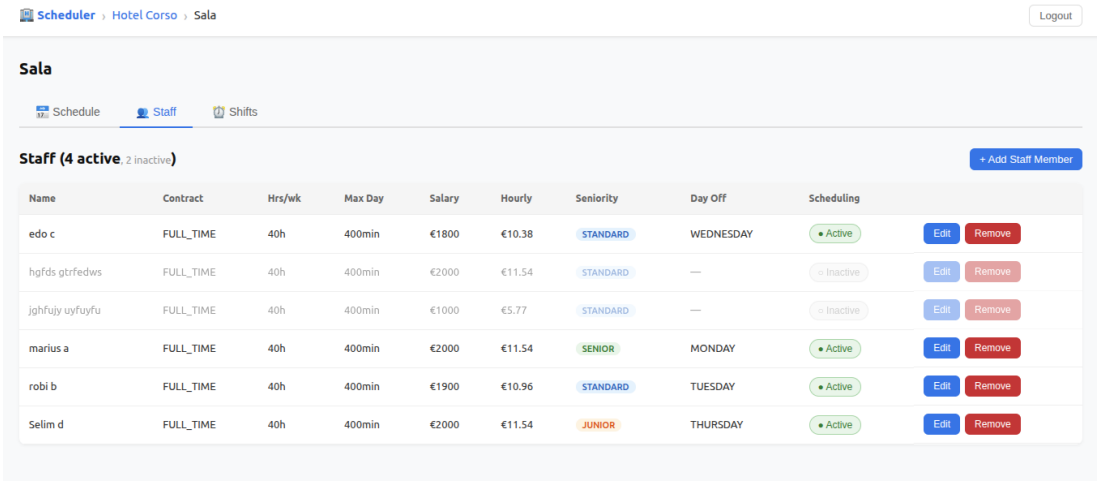


Figure 5. The Staff tab, showing the department's staff pool with contract and seniority details.

The **Shifts tab**, shown in Figure 6, manages shift templates: name, type, times, occupancy metric binding, guests per worker ratio, and default headcount.

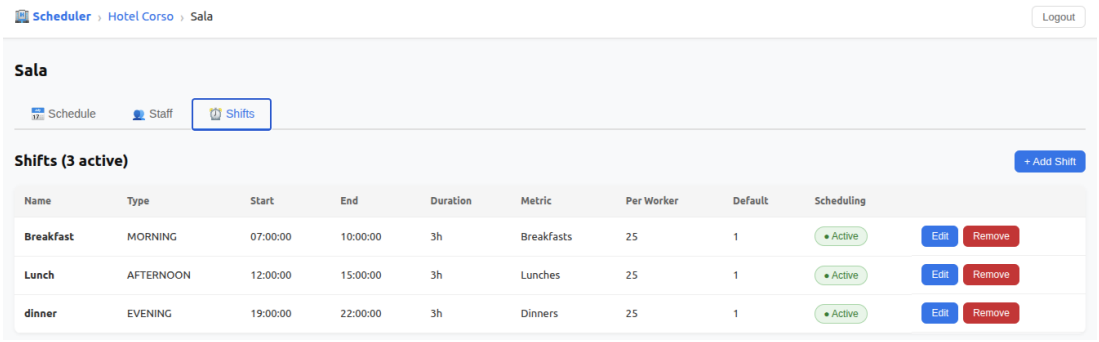


Figure 6. The Shifts tab, showing shift template configuration including the occupancy metric binding.

At the structure level, two additional tabs are available. The **Occupancy tab**, shown in Figure 7, handles PMS data

import: the manager uploads their export file, the AI extraction runs, the preview is shown with amber highlights, and the manager saves. A history table shows all previously saved occupancy records.

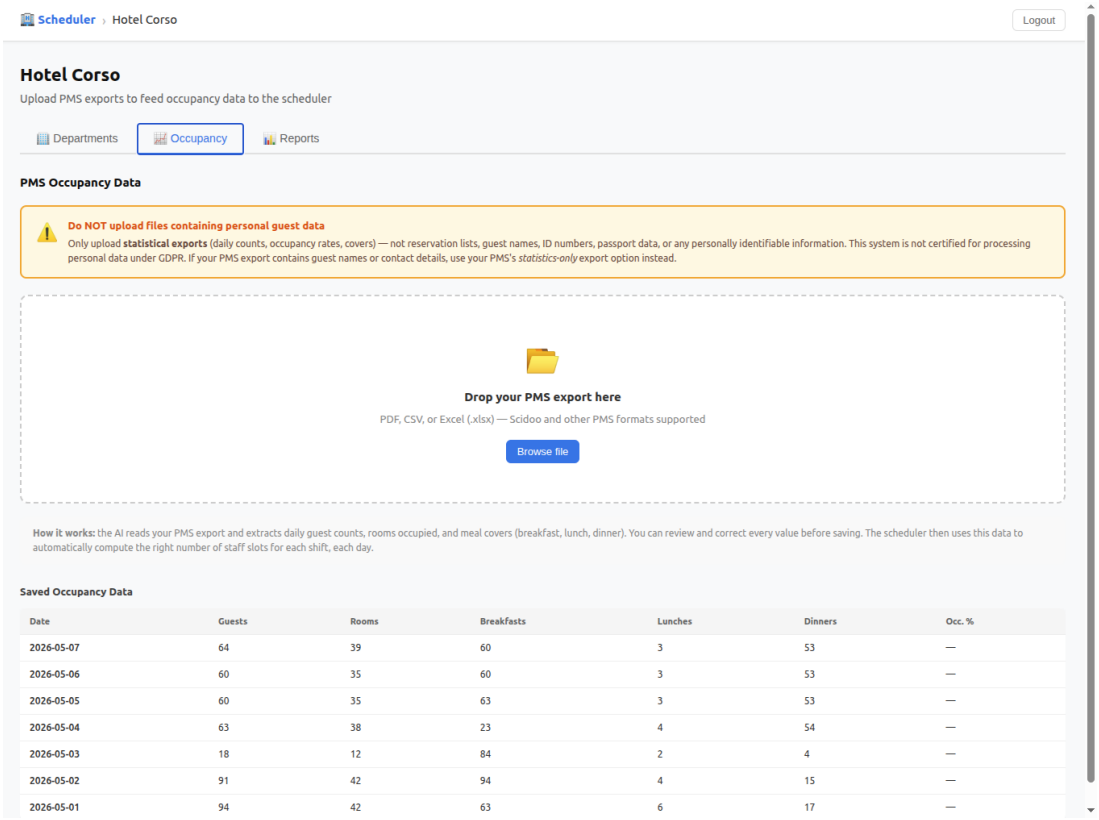


Figure 7. The Occupancy tab, showing the AI extraction preview with amber highlights on uncertain fields.

The **Reports tab**, shown in Figure 8, provides the monthly payroll reporting view described in Section 4.9.

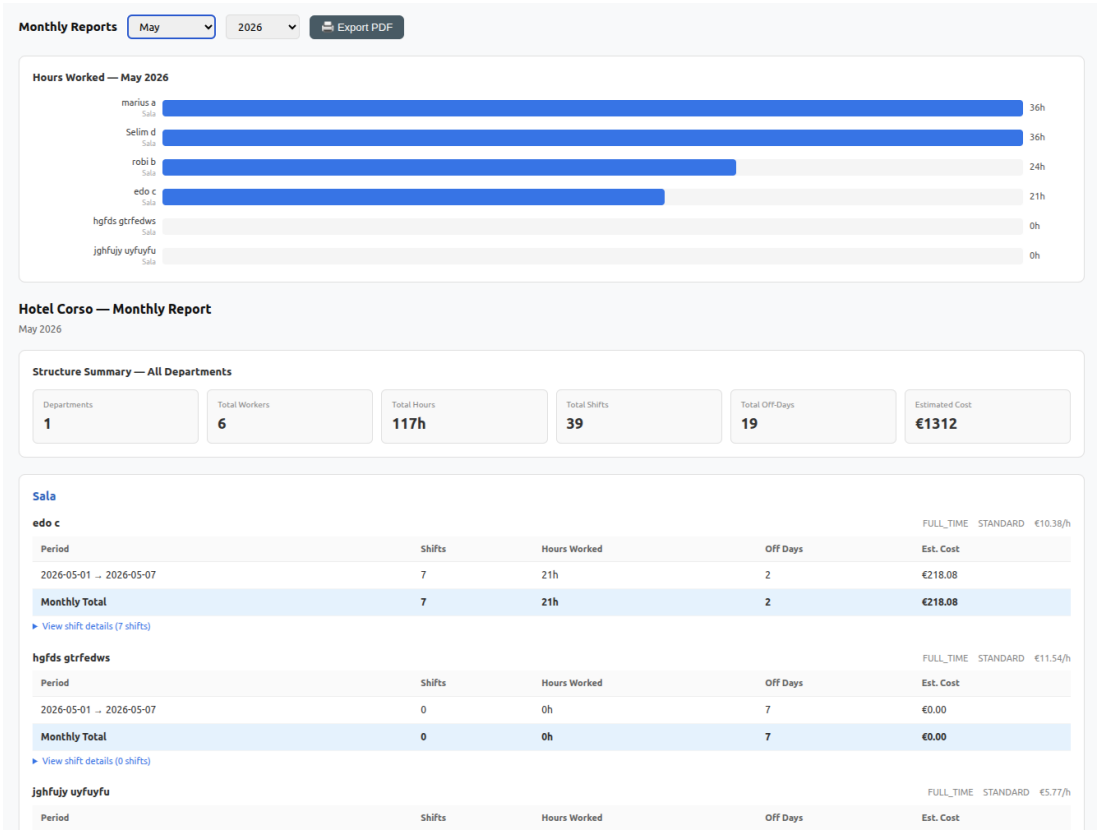


Figure 8. The Reports tab, showing the monthly hours chart and per-department breakdown.

The frontend uses plain CSS with no component library, keeping the dependency footprint minimal and the visual language clean and functional.

4.11 Development Process

Development followed two sprints. The first sprint built the foundation: the domain model, the SpringBoot application, the MongoDB connection, the Timefold integration, the core hard constraints and the initial React frontend with structure/department/schedule navigation and the Staff and Shifts tabs. By the end of the first sprint a manager could enter staff and shift data and generate a schedule.

The second sprint added the occupancy engine, the AI-powered PMS import via Spring AI and the Anthropic API, the `WorkerWeeklySnapshot` record-keeping layer, the reporting tab, the soft constraints and the deployment configuration. The frontend was also substantially refined during this sprint: the schedule grid in particular required several iteration to reach the Staff × Day layout that makes the week readable at a glance.

The codebase is a single repository with a `backend/` directory for the Spring Boot application and a `frontend/` directory for the React application. The backend builds with Maven and deploys on Render, while the frontend builds with Vite and deploys as static assets on Netlify.

5 Evaluation

This chapter describes the evaluation of the prototype. Five participants completed an independent self-administered evaluation using the deployed application, filling in a structured feedback form covering the System Usability Scale, feature relevance ratings, a comparative question on schedule quality, and open-ended qualitative feedback. The chapter covers the evaluation goals (Section 5.1), the System Usability Scale (Section 5.2), the feedback instrument design (Section 5.3), the self-administered evaluation design (Section 5.4), the participants (Section 5.5), the results (Section 5.6), and a discussion of the findings (Section 5.7).

5.1 Evaluation Goals

The evaluation was designed to answer three questions.

First: is the system usable? Hotel managers have, in most cases, never used dedicated scheduling software. If the interface requires training, technical knowledge, or patience to navigate, it will not be adopted regardless of how capable the underlying scheduling engine is.

Second: are the right features implemented? The feature set was derived from interview data, but the analysis step is fallible. Feature relevance ratings provide a direct check on the prioritisation decisions made during requirements clustering.

Third: would someone actually use this? Adoption intention is the roughest but most practically significant question: would this person change their real workflow to include this tool?

5.2 The System Usability Scale

The System Usability Scale (SUS) is a ten item questionnaire for measuring perceived usability [12]. Each item presents a statement about the system rated on a five point Likert scale from “strongly disagree” (1) to “strongly agree” (5). Items alternate between positive and negative framings to prevent response patterning. The raw scores are converted to a 0–100 composite: for odd numbered (positive) items the contribution is the response minus one; for even numbered (negative) items it is five minus the response; the sum of all ten contributions is multiplied by 2.5.

SUS was chosen because it requires no prior knowledge of the system, takes two to three minutes to complete, produces a single interpretable number, and has been empirically validated across hundreds of systems and domains [6]. Its wording is fixed and must not be modified, as the psychometric properties depend on the exact phrasing [12]. Bangor et al. [5] provide the adjective benchmarks: above 80.3 is “Excellent”, 68–80.3 is “Good”, 68 is the industry average, and below 68 indicates increasing usability problems.

5.3 Feedback Instrument Design

The feedback instrument was structured as a Google Form in four parts.

Part 1, pre-demo registration. Collected anonymised background information: current role, experience with staff scheduling, property type and size, current scheduling method, and PMS usage. These establish participant context and allow SUS scores to be interpreted accordingly.

Part 2, demo walkthrough. A structured script guiding participants through the system independently, covering all major features in a realistic workflow sequence, described in Section 5.4.

Part 3, post-demo questionnaire. Three sections.

Section B, SUS. The standard ten items (Q7 to Q16 in form numbering), each on a five point Likert scale, presented in the validated order with no wording modifications.

Section C, feature relevance. Five questions (Q17 to Q21) rating the usefulness of each core feature on a five point scale from "Not useful at all" to "Extremely useful":

- Q17, automatic schedule generation based on staff constraints
- Q18, automatic headcount calculation based on PMS occupancy data
- Q19, AI powered extraction of data from PMS exports (PDF or CSV)
- Q20, manual override of individual shift assignments
- Q21, monthly payroll report with PDF export

Section D, comparative and open feedback. Q22 compares generated schedule quality to the participant's current manual approach. Q23 asks which single feature was most valuable. Q24 asks for the most important missing feature or improvement. Q25 asks about adoption intention.

Part 4, SUS scoring guide. For the researcher only.

5.4 Evaluation Design: Self-Administered Sessions

The evaluation was conducted as a fully self-administered process. Participants received a link to the deployed application and a Google Form containing the complete evaluation instrument. They explored the system independently, following a structured nine step walkthrough script embedded in the form, and completed the feedback questionnaire at the end without researcher involvement or guidance.

The nine steps of the walkthrough script were the following.

1. Complete the pre-demo registration form.
2. Explore the home screen and create a hotel structure.
3. Add a department matching their own operational context.
4. Add staff members with realistic contract data, seniority, and shift preferences.
5. Configure shift templates with occupancy metric bindings and guests per worker ratios.
6. Upload a sample PMS export file and review the AI extraction preview.
7. Create a schedule period and trigger automatic schedule generation.
8. Try the manual override mechanism on at least one shift assignment.
9. View the structure level reports tab and the PDF export.

The self-administered format is methodologically more demanding than a researcher-led walkthrough: participants must navigate the system without guidance, which means friction points in the interface surface naturally rather than being smoothed over by researcher assistance. This makes the results more ecologically valid as a measure of real-world usability, at the cost of some comparability with guided evaluation formats.

5.5 Participants

Five participants completed the evaluation. Table 3 summarises their profiles from the pre-demo registration form.

	P1	P2	P3	P4	P5
Role	External experience	F&B Manager	GM	GM	Front Office Mgr
Schedules	Involved	Regularly	Regularly	Regularly	Regularly
Property	—	Other	Medium	Small	Medium
Current tool	—	Paper	Excel	Paper	Excel
PMS	Unsure	No	Yes	Yes	Yes
Staff per week	1–5	6–15	>50	1–5	16–30

Table 3. Participant profiles from the pre-demo registration form.

The participant pool covers a useful range of roles and operational contexts: from a small independent hotel scheduling fewer than five workers (P4) to a general manager responsible for over fifty seasonal staff (P3). Four of the five participants have direct, regular scheduling responsibility. Three use or have access to a PMS. Two currently schedule on paper, two use Excel.

5.6 Results

5.6.1 SUS Scores

Table 4 shows the raw SUS responses for each participant and the resulting score.

	Q7	Q8	Q9	Q10	Q11	Q12	Q13	Q14	Q15	Q16	SUS	Rating
P1	5	1	5	4	5	1	5	1	5	1	**92.5**	Excellent
P2	5	1	4	1	5	1	3	1	5	1	**92.5**	Excellent
P3	4	1	4	2	5	1	4	1	5	4	**82.5**	Excellent
P4	1	1	4	5	2	1	3	2	1	4	**45.0**	Unacceptable
P5	5	1	4	2	4	1	3	1	4	3	**80.0**	Good
Mean											**78.5**	
SD											**19.6**	

Table 4. Raw SUS responses and computed scores. Even numbered items are negatively phrased; lower responses indicate better usability on those items. Adjective ratings from Bangor et al. [5].

The mean SUS score is **78.5 ± 19.6**. According to the Bangor et al. adjective scale [5], this falls in the **Good** tier, above the industry average of 68. However, the high standard deviation reflects substantial variance between participants: four participants produced scores in the Good to Excellent range (80.0–92.5), while one participant (P4) produced a score of 45.0, classified as Unacceptable. The P4 result is discussed in detail in Section 5.7.

5.6.2 Feature Relevance

Table 5 shows feature relevance ratings per participant and mean scores.

Feature	P1	P2	P3	P4	P5	Mean
Q17, automatic schedule generation	5	5	5	2	4	**4.20**
Q18, occupancy-based headcount	5	5	5	5	4	**4.80**
Q19, AI powered PMS import	5	3	3	5	5	**4.20**
Q20, manual override	5	5	5	3	5	**4.60**
Q21, monthly payroll report	5	5	4	5	5	**4.80**

Table 5. Feature relevance ratings per participant (1 = Not useful at all, 5 = Extremely useful).

Four of the five features received mean scores above 4.0. The occupancy-based headcount engine (Q18) and the monthly payroll report (Q21) tied for the highest mean at 4.80. Automatic schedule generation (Q17) and AI powered PMS import (Q19) both received 4.20, with P4 being the only participant to rate auto generation below 4. Manual override (Q20) scored 4.60, with P4 again the outlier at 3.

5.6.3 Schedule Quality Comparison

P1 selected “Cannot judge” (does not currently schedule staff). P2, P3, and P5 each selected “Slightly better than I could do manually”. P4 selected “Much better than I could do manually”, the most positive response on this item, and notably more positive than P4’s SUS score would suggest.

5.6.4 Adoption Intention

P1 selected “Yes, definitely”. P2 and P5 selected “Probably yes”. P3 selected “Unsure”. P4 selected “Probably not”.

5.6.5 Qualitative Feedback

Three participants provided improvement suggestions. P1 requested an in-system AI assistant, FAQ, or onboarding guide. P2 requested automatic weekly schedule regeneration on a configurable day, so the manager only needs to review and export. P5 requested a direct PMS connection to eliminate the manual file upload step. P3 and P4 left the improvement field blank.

For most valuable feature (Q23), automatic schedule generation was selected by P1, P3, and P4. The monthly payroll report was selected by P2 and P5. Occupancy-based staffing was selected by P2 as a second choice.

5.7 Discussion

5.7.1 The Main Cohort: Four Participants in the Good to Excellent Range

Setting P4 aside for a moment, the four remaining participants produced SUS scores of 80.0, 82.5, 92.5, and 92.5, with a mean of 86.9. These participants span the intended target group: a restaurant manager scheduling on paper, a hotel general manager with 50+ staff using Excel, a front office manager at a medium property, and an external evaluator with scheduling experience. Across this cohort, the system is rated between Good and Excellent, above the industry average by a substantial margin, and three of the four expressed probable or definite adoption intention.

The item-level patterns within this cohort are informative. Q8 (unnecessarily complex) and Q12 (too much inconsistency) received the minimum response of 1 from every participant across all five, meaning nobody found the system complex or inconsistent. Q9 (easy to use) and Q15 (felt confident) were consistently high. The items that pulled scores down were Q10 (need a technical person), Q13 (learn quickly), and Q16 (needed to learn a lot), pointing consistently to onboarding and initial configuration as the area of greatest friction. P1 independently identified this with the suggestion for an AI assistant or onboarding guide. P3’s lower score (82.5) is driven primarily by Q10 and Q16, the same pair.

5.7.2 P4: A Useful Outlier

P4’s score of 45.0 is the result that demands the most careful analysis. P4 is a hotel general manager at a small independent property (fewer than 30 rooms) scheduling one to five staff, currently using pen and paper.

The paradox in P4’s responses is striking: they rated the generated schedule as much better than what they could produce manually (the most positive Q22 response in the sample), yet felt unable to use the system confidently (Q15 $= 1$), doubted they could use it without technical support (Q10 $= 5$), and would probably not adopt it (Q25). They also rated automatic schedule generation, the feature that apparently produced a superior schedule, at only 2 out of 5 for usefulness.

The most coherent interpretation of this profile is a setup cost problem rather than a scheduling quality problem. The system produced a better schedule than P4 could manually, but getting to that output required configuring structures, departments, staff profiles, shift templates with occupancy metrics, and occupancy data, a one-time setup investment that is disproportionate for a property scheduling one to five people. For a hotel of this size, the marginal value of constraint-based scheduling is low, and the configuration overhead is not justified by the weekly time saved.

This is a genuine scope limitation of the prototype: it is designed for SME hotels with enough scheduling complexity to make the setup worthwhile, typically ten to fifty staff across multiple departments. For very small properties, the value proposition does not hold in its current form. This finding is consistent with what the academic literature notes about scheduling tool adoption: the threshold at which dedicated tools become beneficial depends on workforce size and shift complexity [23, 20].

Excluding P4 from the mean is not methodologically appropriate, since the result is real and informative, but reporting the mean with and without P4 is useful: the overall mean is 78.5 (Good), and the mean for properties with six or more staff is 86.9 (Excellent). This distinction maps directly to the system’s intended scope.

5.7.3 Feature Relevance Interpretation

The occupancy-based headcount engine (Q18, mean 4.80) and the monthly payroll report (Q21, mean 4.80) are the joint highest rated features. Both were strongly validated in the interviews and both address pain points described explicitly and consistently across the interview cohort. The payroll report in particular, rated 5 out of 5 by four of the five participants, confirms that the end of month reconciliation problem is real, widespread, and not adequately addressed by any existing tool in the participants' current workflows.

The AI powered PMS import (Q19, mean 4.20) was rated 5 out of 5 by three participants including P4 and P5, both of whom use a PMS and would directly benefit from format-agnostic data ingestion. P2 and P3 rated it at 3, consistent with the earlier finding that its value is most apparent to users who already experience the manual transcription pain point.

The manual override (Q20, mean 4.60) scored highly across the cohort, which directly validates the mixed-initiative design stance [2]. Even P4, the most critical participant overall, rated it at 3 rather than 1.

5.7.4 Adoption Intention and Schedule Quality

Three of five participants expressed probable or definite adoption intention. Four of five, including P4, rated the generated schedule quality as better than their current manual approach, with P4 rating it as much better. The combination of P4's positive schedule quality rating and negative adoption intention reinforces the setup cost interpretation: the scheduling output is valued, but the path to getting there is not.

P5's suggestion of a direct PMS connection and P2's request for automatic weekly regeneration both point to the same underlying desire: reduce the weekly activation cost of using the system, so that the manager's role is review and approve rather than configure and generate.

5.7.5 Limitations

The sample of five participants, while more informative than a single pilot session, remains small and was drawn from the researcher's personal and professional network. The self-administered format, while more ecologically valid than a researcher-led walkthrough, also means there is no way to distinguish between genuine usability difficulties and participant disengagement or misunderstanding of specific form questions. The high standard deviation (19.6) makes the mean sensitive to individual outliers. A larger and more diverse sample would be needed to draw statistically robust conclusions about usability across the full population of SME hotel managers.

6 Conclusion

This project started from a simple observation: hotel managers need better scheduling tools, and the tools that exist are not suitable for them. It tried to take that observation seriously, not by proposing a new algorithm or reviewing the literature and calling that a contribution, but by going to talk to the people who actually build hotel schedules every week, finding out exactly what they need and why, and then building something that addresses it. The chapter summarises the contributions (Section 6.1), reflects on the approach taken (Section 6.2), discusses the limitations of the current prototype (Section 6.3), outlines future work (Section 6.4), and closes with final remarks (Section 6.5).

6.1 Summary of Contributions

The project produced four interconnected contributions.

A structured requirements analysis. Six semi-structured interviews with hotel managers were transcribed and tagged using a ten category coding scheme grounded in requirements engineering methodology. The process produced approximately forty candidate features, clustered into five implementation priorities and three explicit design principles. Every feature in the prototype can be traced back to specific tagged interview passages, giving the specification a level of traceability that goes beyond typical thesis level requirements work.

A tool survey establishing why the gap exists. The scheduling technology exists. The problem is not that no one has built constraint solvers or workforce management platforms, it is that all existing solutions fail SME hotels in the same ways: cost, forced integration, automation that removes manager agency, and shift models that do not match how hospitality actually operates. The survey documented this mismatch in detail and provided the design brief for the prototype.

A working prototype. A Spring Boot backend with Timefold constraint solving, a React frontend, and MongoDB Atlas storage, deployed publicly on Render and Netlify. The prototype implements automatic constraint-based schedule generation, an occupancy engine that derives required headcount from expected guest volume via a configurable

guests per worker model, an AI powered PMS import using Spring AI and the Anthropic Claude API, a manual override mechanism with full transparency in the UI, a five state schedule life cycle, and a monthly record-keeping layer that automatically maintains per-worker hour and cost records on finalisation and exports them as a PDF for payroll handover.

A usability evaluation. Five participants ranging from a restaurant manager scheduling on paper to a hotel general manager responsible for over fifty seasonal staff completed an independent self-administered evaluation. The mean SUS score was **78.5 ± 19.6**, placing the overall result in the Good tier of the Bangor et al. adjective scale [5]. Four of the five participants produced scores in the Good to Excellent range (80.0–92.5); the outlier (P4, 45.0) was a general manager of a very small property scheduling one to five staff, whose response profile pointed to a setup cost problem rather than a scheduling quality problem. P4 rated the generated schedule as much better than manual but found the configuration overhead disproportionate for their scale. Excluding P4, the mean for participants at properties with six or more staff was 86.9, in the Excellent tier. Three of five participants expressed probable or definite adoption intention.

6.2 Reflections on the Approach

The decision to start from requirements rather than from technology was a positive one. The investment in interviews, transcription, tagging, and manual clustering produced a specification grounded in real user needs with explicit traceability to evidence. Every development decision had a clear answer to the question of why it was being built, and the evaluation results validate that the right things were built: the occupancy engine and the payroll report, the two features most directly derived from consistently repeated interview pain points, were the joint highest rated features in the evaluation at 4.80 out of 5.00.

The 80/20 automation principle shaped almost everything about the frontend. Implementing the override mechanism, the manual flag, and the visual distinction between solver-generated and manager-corrected shifts added significant complexity. But the evaluation confirmed the importance of this investment: the manual override scored 4.60 out of 5.00 across all five participants, including a perfect score from four of them. It is not a convenience feature but a prerequisite for adoption, and treating it as such was the right call.

The P4 result is the most instructive output of the evaluation, precisely because it is not a success. A hotel general manager who rated the generated schedule as much better than manual but declined to adopt the system is telling you something specific: the scheduling quality is not the bottleneck, the onboarding is. This is actionable design feedback that a uniformly positive evaluation would not have produced. The minimum viable property size for the current prototype is somewhere around ten to fifteen staff. Below that, the configuration overhead exceeds the weekly time saved, and the tool is not justified. This is a scope constraint worth stating clearly in the product definition, not a flaw in the system's scheduling logic.

The self-administered evaluation format, while more demanding for participants than a researcher-led walkthrough, produced more ecologically valid results. Participants discovered friction points, particularly in initial configuration, that a guided session would have concealed. The Q10 and Q16 responses across the cohort, needing technical support and needing to learn a lot, point consistently to a gap in onboarding support that is now clearly defined and addressable.

6.3 Limitations

The prototype has real limitations that should not be minimised.

It is a proof of concept, not a production-ready system. There is no multi-tenant isolation, no role-based access control, and no per-user session management. A production version would need all of this before it could handle real hotels with real staff data.

The constraint model covers the most important rules identified in the interviews but not all of them. Fairness-based rotation, ensuring that undesirable shifts are distributed equitably over time [28], was identified as a requirement but not implemented. Competency and skill tagging was also deferred. These are features interviewees mentioned explicitly and would notice in a real deployment.

Manual overrides are not currently preserved across a re-solve of the same schedule period: regenerating a schedule rebuilds all assignment slots from scratch, discarding any prior manager corrections. A mechanism for pinning overridden assignments through subsequent solves is a near-term priority, not a deep architectural change.

The solver runs synchronously. For a large hotel with many departments or long planning horizons this would block the API and time out. An asynchronous solver with a progress callback is necessary for any production deployment.

The prototype is not well suited to very small properties, with fewer than ten staff. The evaluation confirmed this: the general manager with the smallest property produced the lowest SUS score and declined adoption, not because

the scheduling output was poor (they rated it as much better than manual) but because the configuration overhead was disproportionate.

The evaluation sample of five is small, geographically limited, and drawn from the researcher's professional network. The high standard deviation (19.6) reflects genuine variation in user context rather than noise, but a larger and more diverse sample would be needed for statistically robust conclusions.

6.4 Future Work

The next steps follow directly from the evaluation findings and the known limitations.

Onboarding and in-system guidance. The clearest signal from the evaluation is that the configuration steps need better in-system support. An interactive onboarding flow, contextual tooltips for the occupancy metric fields, and an AI powered setup assistant would directly address the Q10 and Q16 responses across the cohort and P1's explicit suggestion. This is the highest priority UX improvement.

Automatic weekly schedule regeneration. P2's suggestion, a configurable auto generate day where the system generates the next week's schedule and the manager reviews and approves, is a natural extension of the 80/20 automation principle. It would significantly reduce the weekly activation cost and make the tool more viable as a routine workflow component.

Direct PMS integration. P5's suggestion for a direct PMS API connection (Opera, Mews, Apaleo, Scidoo) would eliminate the file upload step entirely, making the occupancy data flow automatic. The current AI import is a pragmatic format-agnostic solution that works for any PMS; a native integration would be the production-quality next step. A counter argument worth noting is that many PMS platforms run on proprietary software with closed or restricted APIs, so development in this direction needs more feedback data and, eventually, careful planning around which integrations are commercially and technically feasible.

Simplified mode for small properties. The P4 result suggests a genuine need for a lighter configuration path for very small properties. A quick setup mode that skips occupancy-based headcount and generates a simple constraint-satisfying weekly schedule from just a staff list and shift definitions would make the tool accessible to a wider range of property sizes.

Multi-tenant authentication. Per-hotel accounts with role-based access are a prerequisite for any real deployment. The domain model is already structured for multi-tenancy via `structureId` scoping; the missing piece is the authentication and authorisation layer.

Extended constraint model. Fairness-based rotation, competency tagging, and more granular labour law rule support were all identified in the requirements analysis and should be the first backend implementation priority after the current prototype.

Larger evaluation. A properly powered evaluation, with more participants, geographically and operationally diverse, and a standardised task-completion protocol, would provide statistically more robust usability evidence and allow meaningful segmentation of results by property size, staff count, and current scheduling method.

6.5 Final Remarks

This project was built on a simple bet: that the gap between what hotel managers need and what the market offers is not a technology problem, but a design problem. The algorithms exist. The solvers exist. What was missing was a system shaped around the actual workflow of a small hotel, one that speaks the language of occupancy, covers, and seniority rules, that does the mechanical work without taking away the manager's judgement, and that does not require adopting five other modules just to access a schedule generator.

The evaluation results, imperfect sample size and all, suggest the bet was worth making. Where the system fits the user's context, it scores well and earns adoption intent. Where it does not fit, as with a five-person property where the setup cost exceeds the weekly benefit, it tells you that clearly too. Both outcomes are informative. Both point toward what a second version should do differently.

Project Artifacts

All materials produced in the course of this project are collected in a private GitHub repository, available to the thesis supervisor and evaluation committee upon request. The repository contains the following.

- **Codebase.** The full source code of the prototype, organised as a monorepo with a `backend/` directory (Spring Boot, Java 21, Timefold Solver, Spring AI) and a `frontend/` directory (React, Vite). Build and deployment configuration for Render (backend) and Netlify (frontend) is included.

- **Interview transcripts.** The six interview transcripts retained for analysis, in their original Italian, with full line-by-line tagging and English-language annotation notes for each tagged item.
- **Tagging guide.** The coding scheme document defining all ten tag types, their definitions, and domain-specific examples, produced before analysis began.
- **Tool survey.** The standalone LaTeX document surveying existing scheduling tools across the three product categories examined in Section 3.
- **Evaluation package.** The full evaluation instrument, including the pre-demo registration form, the self-administered demo walkthrough script, the post-demo SUS questionnaire, and the SUS scoring guide.
- **Feature list.** The approximately forty candidate features produced by the AI-assisted extraction step, with explicit links to the tagged interview passages that motivated each one, and the clustering and prioritisation notes.

References

[1] C. Abras, D. Maloney-Krichmar, J. Preece, et al. User-centered design. *Bainbridge, W. Encyclopedia of Human-Computer Interaction. Thousand Oaks: Sage Publications*, 37(4):445–456, 2004.

[2] J. E. Allen, C. I. Guinn, and E. Horvitz. Mixed-initiative interaction. *IEEE Intelligent Systems and their Applications*, 14(5):14–23, 1999.

[3] B. Y. Ang, S. W. S. Lam, Y. Pasupathy, and M. E. H. Ong. Nurse workforce scheduling in the emergency department: A sequential decision support system considering multiple objectives. *Journal of nursing management*, 26(4):432–441, 2018.

[4] R. Bai, E. K. Burke, G. Kendall, J. Li, and B. McCollum. A hybrid evolutionary approach to the nurse rostering problem. *IEEE Transactions on Evolutionary Computation*, 14(4):580–590, 2010.

[5] A. Bangor, P. Kortum, and J. Miller. Determining what individual SUS scores mean: Adding an adjective rating scale. *Journal of Usability Studies*, 4(3):114–123, 2009.

[6] A. Bangor, P. T. Kortum, and J. T. Miller. An empirical evaluation of the System Usability Scale. *International Journal of Human-Computer Interaction*, 24(6):574–594, 2008.

[7] M. Bano, D. Zowghi, A. Ferrari, P. Spoletini, and B. Donati. Teaching requirements elicitation interviews: an empirical study of learning from mistakes. *Requirements Engineering*, 24(3):259–289, 2019.

[8] T. Baum, C. Cheung, H. Kong, A. Kralj, S. Mooney, H. Nguyễn Thị Thanh, S. Ramachandran, M. Dropulić Ružić, and M. L. Siow. Sustainability and the tourism and hospitality workforce: A thematic analysis. *Sustainability*, 8(8):809, 2016.

[9] L. Benhadda, M. N. Chibili, S. De Bruyn, C. Lashley, S. Penninga, and B. Rowson. *Modern hotel operations management*. Noordhoff Uitgevers, 2016.

[10] H. Beyer and K. Holtzblatt. *Contextual Design: Defining Customer-Centered Systems*. Morgan Kaufmann, 1998.

[11] L. A. Booker, J. Mills, M. Bish, J. Spong, M. Deacon-Crouch, and T. C. Skinner. Nurse rostering: understanding the current shift work scheduling processes, benefits, limitations, and potential fatigue risks. *BMC nursing*, 23(1):295, 2024.

[12] J. Brooke. SUS: A "quick and dirty" usability scale. In P. W. Jordan, B. Thomas, B. A. Weerdmeester, and I. L. McClelland, editors, *Usability Evaluation in Industry*, pages 189–194. Taylor & Francis, London, 1996.

[13] K. A. Butler, J. Zhang, C. Esposito, A. Bahrami, R. Hebron, and D. Kieras. Work-centered design: a case study of a mixed-initiative scheduler. In *Proceedings of the SIGCHI conference on Human factors in computing systems*, pages 747–756, 2007.

[14] B. Cheng, J. H.-M. Lee, and J. Wu. A nurse rostering system using constraint programming and redundant modeling. *IEEE Transactions on information technology in biomedicine*, 1(1):44–54, 2002.

[15] M. Dwivedi, S. Upadhyay, and A. Tripathi. A working framework for the user-centered design approach and a survey of the available methods. *International Journal of Scientific and Research Publications*, 2(4):12–19, 2012.

[16] I. Hadar, P. Soffer, and K. Kenzi. The role of domain knowledge in requirements elicitation via interviews: an exploratory study. *Requirements Engineering*, 19(2):143–159, 2014.

[17] D. Hur, V. A. Mabert, and K. M. Bretthauer. Real-time work schedule adjustment decisions: An investigation and evaluation. *Production and Operations Management*, 13(4):322–339, 2004.

[18] S. Kusluvan. Characteristics of employment and human resource management in the tourism and hospitality industry. *Managing employee attitudes and behaviors in the tourism and hospitality industry*, 1:3–24, 2003.

[19] M. Labidi, M. Mrad, A. Gharbi, and M. Louly. Scheduling it staff at a bank: A mathematical programming approach.

[20] R. J. Malony. Analysis of staff scheduling effect on hospitality staffing service efficiency during high-occupancy conditions using discrete event simulation (des). 2011.

[21] D. Meignan. A heuristic approach to schedule reoptimization in the context of interactive optimization. In *Proceedings of the 2014 Annual Conference on Genetic and Evolutionary Computation*, pages 461–468, 2014.

[22] A. Meisels, E. Gudes, and G. Solotorevsky. Combining rules and constraints for employee timetabling. *International Journal of Intelligent Systems*, 12(6):419–439, 1997.

[23] S. Mohan. Scheduling part-time personnel with availability restrictions and preferences to maximize employee satisfaction. *Mathematical and Computer Modelling*, 48(11-12):1806–1813, 2008.

[24] K. Murakami, S. O. Tasan, M. Gen, and T. Oyabu. A case study of human resource allocation for effective hotel management. *Industrial Engineering and Management Systems*, 10(1):54–64, 2011.

[25] C. M. Ngoo, S. L. Goh, N. R. Sabar, S. Abdullah, G. Kendall, et al. A survey of the nurse rostering solution methodologies: The state-of-the-art and emerging trends. *IEEE Access*, 10:56504–56524, 2022.

[26] B. Nuseibeh and S. Easterbrook. Requirements engineering: a roadmap. In *Proceedings of the Conference on the Future of Software Engineering*, pages 35–46. ACM, 2000.

[27] C. Palomares, X. Franch, C. Quer, P. Chatzipetrou, L. López, and T. Gorschek. The state-of-practice in requirements elicitation: an extended interview study at 12 companies. *Requirements engineering*, 26(2):273–299, 2021.

[28] M. Rocha, J. F. Oliveira, and M. A. Carravilla. Cyclic staff scheduling: optimization models for some real-life problems. *Journal of scheduling*, 16(2):231–242, 2013.

[29] N. Simeunović, I. Kamenko, V. Bugarski, M. Jovanović, and B. Lalić. Improving workforce scheduling using artificial neural networks model. *Advances in Production Engineering & Management*, 12(4):337–352, 2017.

[30] S. F. Smith, O. Lassila, and M. Becker. Configurable, mixed-initiative systems for planning and scheduling. *Advanced Planning Technology*, 98:235–241, 1996.

[31] I. Sommerville. *Software Engineering*. Pearson, 9 edition, 2011.

[32] N. Todorovic and S. Petrovic. Bee colony optimization algorithm for nurse rostering. *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, 43(2):467–473, 2012.

[33] B.-C. Yen. An interactive scheduling agent on the internet. In *Proceedings of the Thirtieth Hawaii International Conference on System Sciences*, volume 4, pages 220–229. IEEE, 1997.

[34] P. Zave and M. Jackson. Four dark corners of requirements engineering. *ACM Transactions on Software Engineering and Methodology*, 6(1):1–30, 1997.